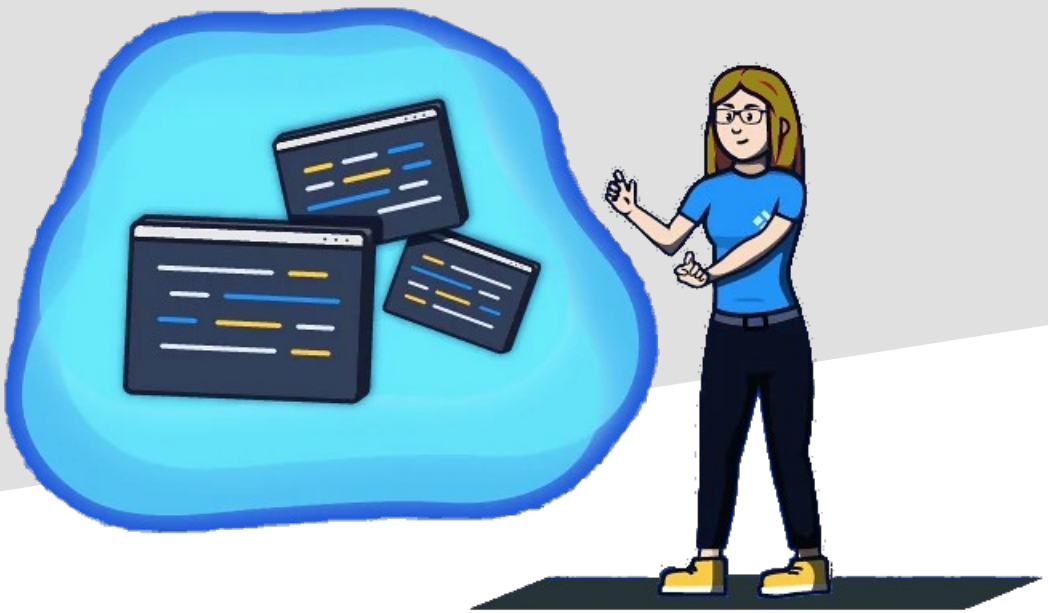
Tema 05. POO avanzada



Índice

1. Sobrecarga de métodos
2. Herencia. Superclase y subclases
3. Sobreescritura de métodos
4. Clases y métodos abstractos
5. Polimorfismo
6. Paquetes
7. Enumerados
8. Interfaces

1.- Sobrecarga de métodos

Un método sobrecargado es un método que tiene el mismo nombre que otro pero con diferentes argumentos.

<u>REGLAS</u>

- Un método puede ser sobrecargado en la misma clase o en una subclase (lo veremos en la herencia).
- La lista de argumentos debe ser diferente (en número y/o en tipos).
- Puede cambiar el tipo de retorno.
- Puede cambiar el modificar de acceso.
- Los constructores también se pueden sobrecargar (ver en “`EjemploSobrecarga.java`”).

2.- Herencia

La herencia es un mecanismo que permite crear clases a partir de otras existentes:

- Heredando y posiblemente añadiendo atributos.
- Heredando y posiblemente añadiendo y/o modificando métodos.

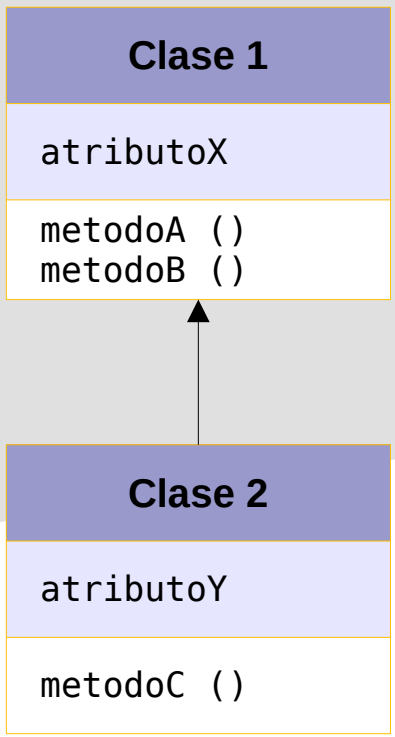
Las clases pueden heredar características de otras clases, lo que permite aumentar la reutilización del software:

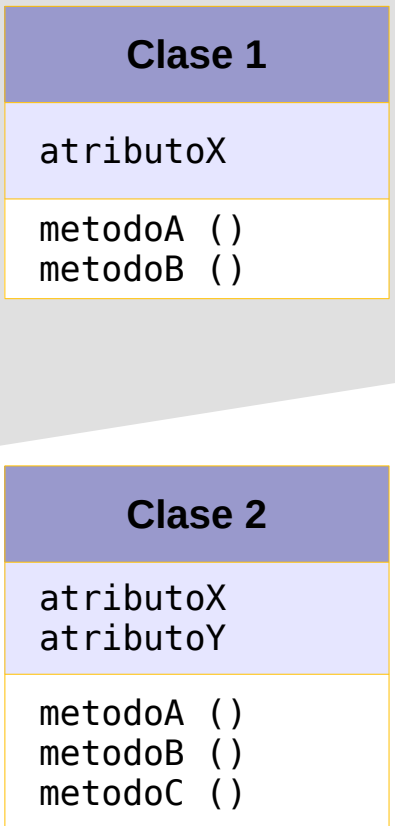
- La clase de la que se hereda se denomina **superclase** o clase padre.
- La clase que hereda se denomina **subclase** o clase hija.

La herencia puede aplicarse muchas veces lo que permite crear una **jerarquía de clases.**

2.- Herencia

EJEMPLO





2.- Herencia

EJEMPLO

En un centro de enseñanza tenemos la clase **Persona**:

- Atributos de persona: `nombre, dni y direccion`.
- Métodos: `setNombre, setDireccion, obtenerDatosPorDni...`

Creamos la clase **Alumno** que hereda de Persona. Además de los atributos y métodos de Persona la clase Alumno tiene:

- Atributos: `curso, notas (array).`
- Métodos: `ponerNotaEnAsignatura, consultarCurso...`

Creamos la clase **Profesor** que hereda de Persona. Además de los atributos y métodos de Persona la clase Profesor tiene:

- Atributos: `especialidad, horario y antigüedad.`
- Métodos: `getEspecialidad, mostrarHorario...`

2.- Herencia

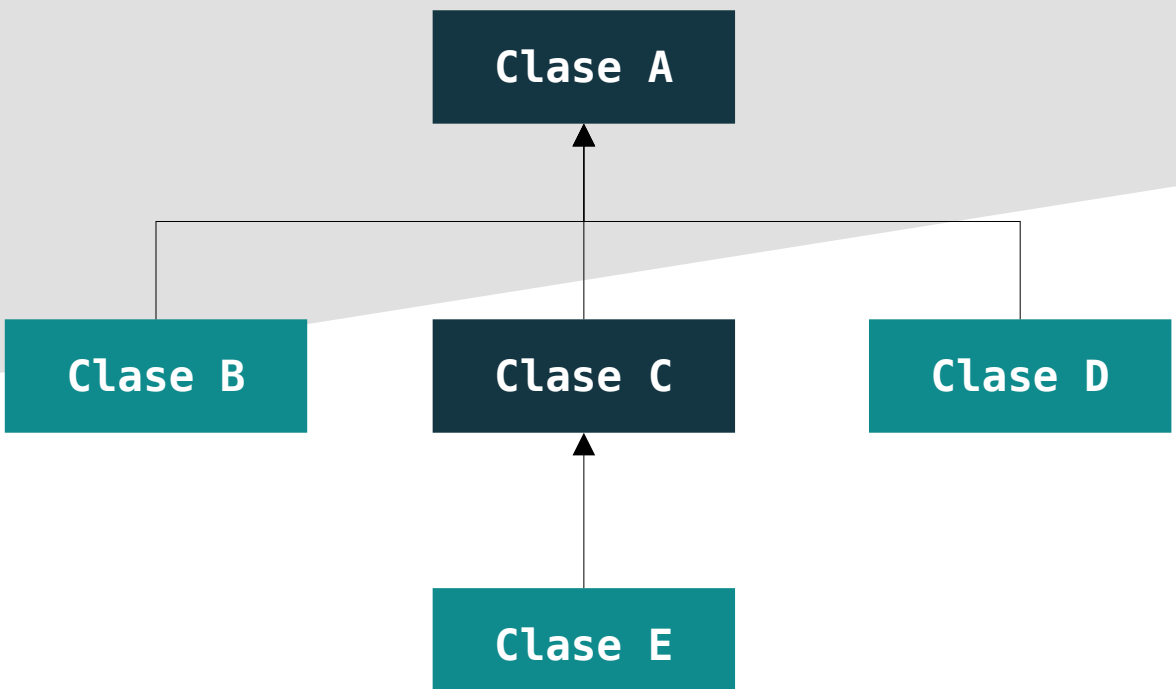
JERARQUÍA

A es la superclase de B, C y D.

C es la superclase de E.

B, C y D son subclases de A.

E es una subclase de C.



2.- Herencia

En Java, una clase sólo puede extender una superclase (herencia simple). Se utiliza la cláusula `extends`:

```
public class NomSubclase extends NomSuperclase{
        // codigo de la subclase
}

public class Clase2 extends Clase1 {
        // código de la Clase 2
}
```

NOTA: Para implementar la herencia múltiple en JAVA se usan las interfaces.

2.- Herencia

EJEMPLO EN JAVA

```
public class Vehiculo {
    private String numSerie;
    private String color;
    // metodos get y set
}

public class Coche extends Vehiculo {
    private int npuertas;
    private int numAirbags;

    public void realizarItv() {
    }
}

public class Camion extends Vehiculo {
    private int pma; // peso maximo autorizado

    public void paradaRegulada() {
    }
}
```

2.- Herencia

VISIBILIDAD

Modificadores de visibilidad de los atributos:

- **`public`**: accesible desde cualquier clase.
- **`protected`**: accesible desde la propia clase y de sus subclases. Ojo: también desde la clase del mismo paquete.
- **`private`**: sólo accesible en la propia clase.
- **Sin modificador**: significa que es accesible desde la propia clase y desde cualquier clase del mismo paquete (se suele llamar visibilidad **friendly** o visibilidad de paquete).

2.- Herencia

VISIBILIDAD

VISIBILIDAD	CLASE	PAQUETE	SUBCLASE	TODOS
public	SI	SI	SI	SI
private	SI	NO	NO	NO
protected	SI	SI	SI	NO
friendly	SI	SI	NO	NO

2.- Herencia

THIS Y SUPER

- **`this`** referencia al objeto actual en el que se está ejecutando el código.

`this`: permite referenciar a los atributos y métodos del objeto actual

`this()`: hace referencia al constructor del objeto actual.

- **`super`** referencia al objeto de la superclase del objeto actual en el que se está ejecutando el código.

`super`: permite referenciar a los atributos y métodos del objeto de la superclase.

`super()`: hace referencia al constructor del objeto de la superclase.

2.- Herencia

THIS Y SUPER (EJEMPLO)

En ocasiones es imprescindible el uso de **this** para resolver ambigüedades.

```
public class Alumno {
    private int edad;
    private String nombre;

    public Alumno(String nombre, int edad) {
        this.edad = edad;
        this.nombre = nombre;
    }
}
```

2.- Herencia

THIS Y SUPER (EJEMPLO)

En ocasiones es imprescindible el uso de **super** para resolver ambigüedades.

```
public class Persona {
    private String nombre;

    public String toString() {
        return "Nombre" + nombre;
    }
}

public class Empleado extends Persona {
    private int sueldo;

    public String toString() {
        return super.toString() + "Sueldo" + sueldo;
        // sin super se llamará a sí mismo de
        // forma infinita (recursividad infinita)
    }
}
```

2.- Herencia

CONSTRUCTORES

Las subclases deben definir su propio constructor.

Normalmente será necesario inicializar los atributos de la superclase. Para ello se llama a su constructor desde el de la subclase.

```
/* Constructor de una subclase */
public Subclase() { //Con parametros si hacieran falta
    // invoca el constructor de la superclase
    super(); //Con parametros si hacieran falta
    // inicializar sus atributos
    //...
}
```

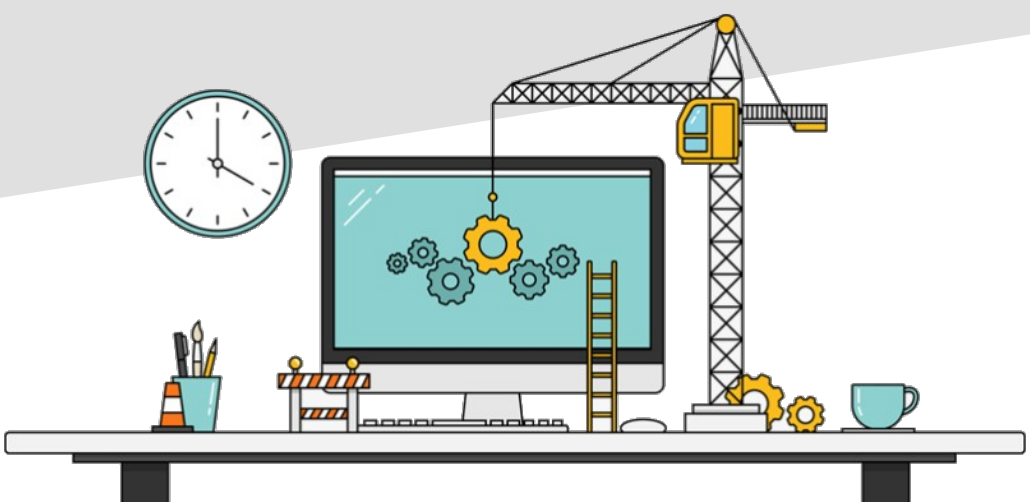
La llamada a “super” debe ser la primera instrucción del constructor de la subclase.

2.- Herencia

CONSTRUCTORES

Si desde un constructor de una subclase no se llama expresamente al de la superclase el compilador añade la llamada al constructor sin parámetros.

En el caso de que la superclase no tenga un constructor sin parámetros se produciría un error de compilación.



3.- Sobreescritura de métodos

Cada vez que se tiene una clase que hereda un método de una superclase, se tiene la oportunidad de **sobreescribir** el método (a menos que dicho método esté marcado como final).

El beneficio de sobreescribir un método heredado es la posibilidad de definir un comportamiento específico para los objetos de la subclase

Ya lo hemos hecho cada vez que programamos el `toString` o el `equals`. Estamos sobreescribiendo el método de la clase `Object`.

3.- Sobreescritura de métodos

REGLAS

- La lista de argumentos del método debe ser la misma.
- El tipo de retorno debe de ser el mismo o un subtipo del tipo de retorno declarado originalmente.
- El nivel de acceso no debe de ser más restrictivo pero puede puede ser menos restrictivo (de más a menos restrictivo el orden es: `private, friendly, protected, public`)
- El método sobrescrito puede lanzar menos excepciones que el método base. No puede lanzar excepciones nuevas.
- No se puede sobreescibir un método marcado como `final` ni marcado como `static`.

3.- Sobreescritura de métodos

EJEMPLO (EjemploSobreescritura.java)

```
public class Rectangulo {
    private double lado1;
    private double lado2;

    // .....
    public void dibujar() {
        System.out.println("Dibujando rectángulo con lados " + lado1 + " " + lado2);
    }

    public double get_lado1() {
        return lado1;
    }
}

public class Cuadrado extends Rectangulo {
    public void dibujar() // Este método está sobreeescrito
    {
        System.out.println("Dibujando cuadrado con lado " + super.get_lado1());
    }
}
```

3.- Sobreescritura de métodos

CLASES Y MÉTODOS FINAL

- Una clase **`final`** es aquella de la que no se podrá heredar. Es decir es una clase que no puede tener ninguna subclase.
- La utilidad de poner una clase como **`final`** es asegurarse que su comportamiento siempre será ese. Ninguna clase podrá heredar de ella y modificar su comportamiento, por lo que proporciona seguridad en ese sentido. Por ejemplo, la clase `String` es final.
- Si un método es `final` no se podrá sobreescribir. Del este modo si queremos que ninguna clase que herede este método cambie su comportamiento lo declararemos como **`final`**.

4.- Clases abstractas

- Una clase **`abstract`** es una clase de la que no se pueden crear objetos. Su utilidad es permitir que otras clases hereden de ella.
- Un método abstracto es un método declarado en una clase para el cual esa clase no proporciona la implementación (el código).
- Una clase abstracta puede contener métodos no abstractos y métodos abstractos. No es obligatorio que tenga un método abstracto.
- Una clase que hereda de una clase abstracta debe implementar los métodos abstractos (escribir el código) o bien volverlos a declarar como abstractos, con lo que ella misma se convierte también en clase abstracta.

4.- Clases abstractas

Se pueden crear referencias a clases abstractas como a cualquier otra clase.

```
Figura f;
```

Sin embargo una clase abstracta no se puede instanciar, es decir, no se pueden crear objetos de una clase abstracta. Error:

```
Figura f = new Figura(); // ERROR
```

Sin embargo utilizando el up-casting (conversión hacia arriba) es posible hacer:

La invocación al método area se resolverá en tiempo de ejecución y la JVM llamará al método de la clase adecuada. En nuestro ejemplo se llamará al método area de la clase Rectangulo.

```
Figura f = new Rectangulo();
f.area();
```

5.- Polimorfismo

Un objeto solamente es de una clase (la que se le asigna cuando se construye ese objeto, es decir cuando se hace el new).

La referencia a un objeto es polimórfica (puede tomar varias formas) porque puede referirse a objetos de diferentes clases. Para que esto sea posible debe haber una relación de herencia entre esas clase.

```
Figura figura;
// Creamos un objeto de tipo Rectangulo, aunque el tipo de la referencia es Figura.
figura= new Rectangulo(4.4, 2.1);
// Ojo: al reves no podría hacerse porque un Rectángulo es una Figura pero NO al contrario
// Circulo circulo= new Figura (....)  ERROR!!!
// Creamos un objeto de tipo Circulo, aunque el tipo de la referencia es Figura
figura=new Circulo(9.7, 6);
```

La combinación de la herencia y el enlace dinámico es lo que se conoce como polimorfismo.

5.- Polimorfismo

OPERADOR INSTACEOF

El operador `instanceof` sirve para saber el tipo de un objeto. Devuelve `true` o `false`.

Es muy útil en casos de herencia, no se debe abusar por ser poco eficiente.

La sintaxis es `objeto instanceOf Clase`

Ejemplo: una clase `ClaseB` que hereda de `ClaseA`

```
ClaseA  a =  new ClaseA();

if(a instanceof ClaseA);//TRUE
if(a instanceof ClaseB);//FALSE
```

5.- Polimorfismo

CONVERSIÓN ENTRE OBJETOS

Un objeto hijo sí es instancia del objeto padre.

Ejemplo: una clase `ClaseB` que hereda de `ClaseA`

```
ClaseA  b =  new ClaseB();

if(b instanceof ClaseA); // TRUE
if(b instanceof ClaseB); // TRUE
```

6.- Paquetes

Las clases se agrupan en paquetes de acuerdo con su funcionalidad. Son independientes de la jerarquía de clases.

La declaración de paquetes debe ser la primera instrucción válida de una clase:

```
1 package paquete;
```

Uso de las clases definidas en un paquete:

```
import paquete.clase;
import paquete.*;
```

Los paquetes relacionados se pueden agrupar en un único paquete. Un ejemplo es el paquete `java` que contiene a otros paquetes como `lang, util, io...`

La convención para los nombres de paquetes es utilizar el nombre del dominio seguido del nombre de la organización. Por ejemplo : `com.sun.eng`

7.- Enumerados

Un tipo enumerado en Java es un tipo restringido a un conjunto de valores.

```
public enum TipoDemarcacion {
    PORTERO, DEFENSA, CENTROCAMPISTA, DELANTERO
}

public enum TipoEstadoCivil {
    SOLTERO, CASADO, VIUDO, DIVORCIADO
}
```

Pueden crearse como una clase independiente o dentro de otra.

Los valores por convención se ponen en mayúsculas.

7.- Enumerados

Crear un atributo de tipo enumerado:

```
final TipoEstadoCivil estadoCivil;
```

Asignar un valor a un enumerado:

```
estadoCivil=TipoEstadoCivil.SOLTERO;
```

Mostrar el valor. El enumerado tiene redefinido su `toString` así que podemos ver el valor de esta forma:

```
System.out.println("El estado civil es " + estadoCivil);
```

Es posible añadir atributos a un tipo enumerado

8.- Interfaces

Interfaz: conjunto de constantes y métodos públicos sin cuerpo (sin código, como si fuesen métodos abstractos).

Se encarga de establecer unas líneas generales sobre el comportamiento (métodos) que deberían tener los objetos de las clases que implementen esa interfaz, es decir, no incluyen el código de las acciones que define.

Muchas interfaces en Java termina con sufijos del tipo "`-able`", "`-or`", "`-ente`" que significan algo así como capacidad o habilidad para hacer o ser receptores de algo (configurable, serializable, modificable, clonable, ejecutable...).

Ejemplo: Interfaz `Arrancable` con un método `arrancar()` que se implementa por clases de tipos de vehículos (`Coche`, `Moto`) pero no por otros (`Bicicleta`).

8.- Interfaces

Antes de Java 8 una interfaz sólo podía incluir métodos no implementados y declaración de constantes. A partir de Java 8 también puede incluir métodos implementados (`default`).

Los atributos de una interfaz son siempre `public`, `static` y `final`.

Los métodos de una interfaz son siempre `public` y `abstract` (salvo que sean `default`).

Una clase puede heredar (`extend`) de una sola clase pero puede implementar varias interfaces.

Ejemplo: Podría existir otra interfaz llamada `MedibleConsumo` con un método `medirConsumo()`. La clase `Coche` puede implementar la interfaz `Arrancable` y `MedibleConsumo`.

Las interfaces en JAVA pueden usarse para implementar la **herencia múltiple**.

8.- Interfaces

DEFINICIÓN

```
[modificadores] interface Nombre [cláusulaExtends] [otraInterface]
{
     [cuerpo]
}
```

Modificadores:

- Visibilidad: `public` (interfaz pública) o nada (interfaz no pública); es decir, igual que las clases.
- `abstract` (no hace falta ponerlo, cualquier interfaz es `abstract` por defecto).

cláusulaExtends: `extends interfaz1, ..., interfazN`

8.- Interfaces

Una clase implementa una interfaz si contiene una implementación para cada uno de los métodos declarados en la interfaz.

También se permite que la clase sólo implemente algunos de los métodos de la interfaz; en este caso, quedarían métodos abstractos sin cuerpo, por lo que la clase se convertiría en abstracta.

Una clase puede implementar varias interfaces, con la condición de que contenga un método público con código concreto por cada uno de los métodos de cada interfaz.

Cuando definimos una clase, indicamos la lista de interfaces que la clase implementa.

8.- Interfaces

```
[modificadores] class Nombre [cláusulaExtends] [cláusulaImplements]
{
  [cuerpo]
}
```

- cláusulaImplements:

```
implements interfaz1, ..., interfazN
```

Las interfaces se diseñan dentro de paquetes, igual que las clases, y tienen las mismas visibilidades (pública y de paquete).

En Java existe una jerarquía de interfaces ya definidas como en el paquete `java.lang: Cloneable, Comparable, Runnable, Serializable.`

Entre las interfaces también puede existir relación de **herencia**.

8.- Interfaces

INTERFACES VS CLASES ABSTRACTAS

Principales Diferencias

Característica	Clase Abstracta	Interface
Métodos con implementación	✅ Sí	⚠️ Desde Java 8 (`default` o `static`)
Métodos abstractos	✅ Sí	✅ Sí (implícitamente abstractos)
Atributos de instancia	✅ Sí	❌ No (solo constantes `static final`)
Constructores	✅ Sí	❌ No
Herencia múltiple	❌ No (una sola clase padre)	✅ Sí (múltiples interfaces)

Tema 05. POO avanzada